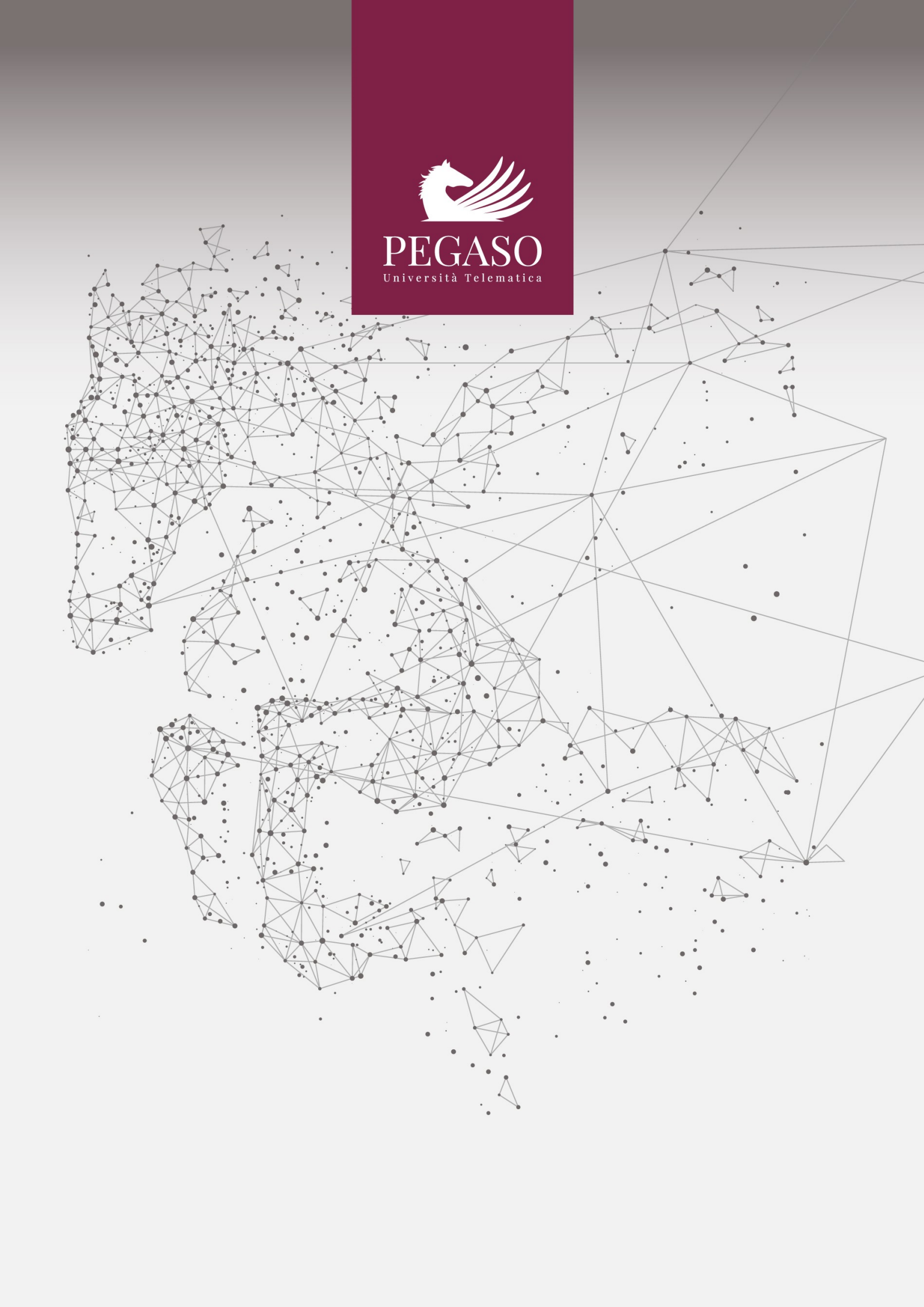

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Indice

1. INTRODUZIONE....3
2. SINCRONIZZAZIONE DEL CLOCK 4
3. CLOCK FISICI....6
BIBLIOGRAFIA 8

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

1. Introduzione

Anche se la comunicazione tra i processi è essenziale, non è l'unico fattore da considerare in un sistema distribuito. Il grado in cui i diversi processi collaborano e si sincronizzano tra loro è strettamente correlato. Il naming, che consente ai processi di condividere risorse o entità in generale, è uno dei meccanismi che contribuisce alla facilitazione della cooperazione.

In questa lezione, il nostro obiettivo principale è introdurre i concetti fondamentali di sincronizzazione e coordinamento delle attività che possono verificarsi tra diversi processi.

Ad esempio, è essenziale garantire che più processi non accedano contemporaneamente alla stessa risorsa condivisa, come potrebbe essere ad esempio un file. Invece, questi processi dovrebbero lavorare insieme per concedere a turno l'accesso esclusivo l'uno all'altro. Un altro esempio potrebbe essere lo scenario in cui più processi devono raggiungere un consenso sulla sequenza in cui si sono verificati alcuni eventi, ad esempio determinare se il messaggio *m1* dal processo *P* è stato inviato prima o dopo il messaggio *m2* dal processo *Q*.

I fenomeni di sincronizzazione e coordinamento sono inestricabilmente legati l'uno all'altro. Quando sincronizziamo i processi, ci assicuriamo che un processo attenda che un altro termini la sua operazione prima di passare a quello successivo. La sfida di gestire la sincronizzazione dei dati consiste nell'assicurarsi che due diversi insiemi di dati siano identici l'uno all'altro.

Lo scopo del coordinamento è quello di gestire le interazioni e le dipendenze che esistono tra le varie attività che compongono un sistema distribuito. Da questo punto di vista, si potrebbe sostenere che il coordinamento è sinonimo di sincronizzazione.

Rispetto al coordinamento in sistemi con un singolo processore o più processori, il processo di coordinamento dei sistemi distribuiti si rivela spesso significativamente più impegnativo. È di fondamentale importanza che un insieme di processi sia in grado di selezionare uno di loro come coordinatore, che può essere realizzato attraverso l'uso di algoritmi di elezione in molti scenari diversi.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

2. Sincronizzazione del clock

Lo scorrere del tempo non è mai ambiguo all'interno di un sistema centralizzato. Una semplice chiamata al sistema operativo è tutto ciò che è necessario affinché un processo ottenga l'ora esatta ogni qual volta ne avesse bisogno. Se un processo *A* richiede il tempo, e poi un po' più tardi anche il processo *B* richiede il tempo, allora il valore che il processo *B* otterrà sarà maggiore il valore ottenuto dal processo *A*. Non c'è nessun modo che possa risultare un valore inferiore. Ad ogni modo, non è facile mettere tutti d'accordo per quanto riguarda il tempo in un sistema distribuito.

Si considerino, come semplice esempio, gli effetti che l'assenza di una sorgente temporale globale ha sul programma Unix *make* per un breve periodo di tempo. I programmi di grandi dimensioni sono tipicamente suddivisi in più file sorgenti in Unix. Questa operazione viene eseguita in modo che se viene apportata una modifica a un file di origine, sarà necessario ricompilare solo quel file anziché tutti i file. I programmatori sono in grado di completare il loro lavoro molto più rapidamente quando non sono tenuti a ricompilare l'intero programma solo perché uno dei suoi file è stato modificato. Un programma tipico può contenere un centinaio di file diversi.

La modalità di funzionamento del *make* è abbastanza semplice e intuitiva. Dopo che il programmatore ha finito di apportare modifiche a tutti i file di origine, esegue *make*, che controlla i tempi in cui tutti i file di origine e i file oggetto sono stati modificati più di recente. Il *make* saprà che il file di origine input.c è stato modificato dalla creazione del file oggetto corrispondente input.o se il timestamp su input.c è ad esempio 2151 e il timestamp su input.o è 2150. Ciò significa che input.c dovrà essere ricompilato. D'altra parte, se output.c ha timestamp 2144 e output.o ha timestamp 2145, allora non è necessario compilare il programma. In questo modo *make* chiama il compilatore per ricompilare i sorgenti dopo aver iterato su tutti i sorgenti per determinare quali devono essere ricompilati.

Si immagini per un momento cosa accadrebbe in un sistema decentralizzato nel caso in cui non esiste un accordo universale sui tempi. Si immagini anche che output.o abbia il tempo 2144 come descritto sopra e che output.c poco dopo viene aggiornato, ma il timestamp rimane a 2143 a causa del fatto che l'orologio sulla sua macchina è leggermente in ritardo, come illustrato nella Figura 1.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

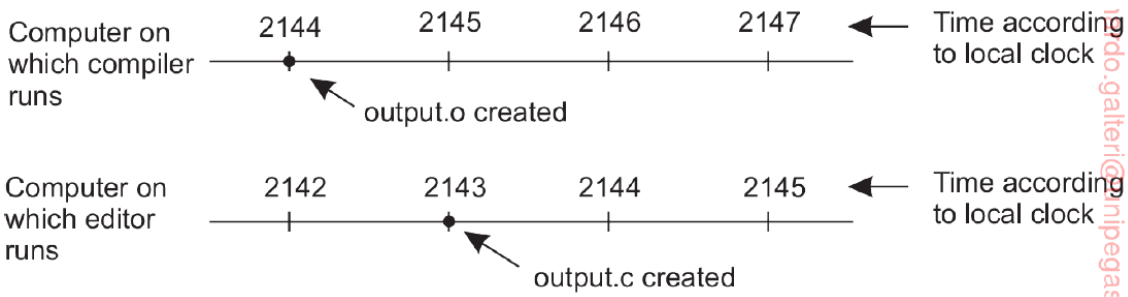


Figura 1 - Courtesy of van Steen, Maarten & Tanenbaum, Andrew. (2017). Distributed Systems, Third edition

Il *make* in questo caso non avvierà il processo di compilazione. Il programma eseguibile binario prodotto come risultato includerà quindi una combinazione di file oggetto derivati sia dai vecchi sorgenti che dai nuovi sorgenti.

Molto probabilmente fallirà e il programmatore impiegherà molto tempo cercando di capire dove ha sbagliato con il codice.

Ci sono un gran numero di altri scenari in cui è richiesta una misura precisa del tempo. L'esempio precedente può essere facilmente adattato per essere applicato ai timestamp dei file in generale con poco sforzo. Quando si considerano domini applicativi come l'intermediazione finanziaria, il controllo della sicurezza ed altri contesti di elevata sensibilità in caso di fallimento, è facile capire perché la misura del tempo accurata è così importante in queste aree.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

3. Clock fisici

Praticamente ogni computer incorpora una sorta di meccanismo per monitorare il passare del tempo. Nonostante il fatto che la parola "clock" sia comunemente usata per riferirsi a questi dispositivi, non sono affatto orologi nel senso tradizionale. Forse la parola "timer" è più appropriata. Nella maggior parte dei casi, un cristallo di quarzo che è stato accuratamente lavorato funge da timer del computer.

I cristalli di quarzo hanno una frequenza ben definita alla quale oscillano quando sono tenuti sotto tensione, e questa frequenza cambia a seconda del tipo di cristallo, di come viene tagliato e della quantità di tensione. Ci sono due registri, un contatore e un registro di mantenimento, che sono collegati ad ogni singolo cristallo. Il contatore viene decrementato dopo ogni oscillazione del cristallo. Quando il contatore raggiunge lo zero, viene generato un interrupt e il registro di mantenimento viene utilizzato per ricaricare il contatore con nuovi valori.

Un timer può essere programmato in questo modo per produrre un interrupt ad una frequenza di sessanta volte al secondo, o a qualsiasi altra frequenza specificata dall'utente. Un "tic" dell'orologio equivale a un interrupt.

Quando il computer viene acceso per la prima volta, normalmente viene richiesto all'utente di immettere la data e l'ora correnti. Queste informazioni vengono quindi convertite nel numero di tic trascorsi da una determinata data di inizio e salvate in memoria. La maggior parte dei computer moderni sono dotati di una RAM CMOS specializzata che viene supportata da una batteria. Di conseguenza, la data e l'ora non devono essere inserite manualmente durante gli avvii successivi. La quantità di tempo attualmente memorizzata in memoria viene aumentata di uno con ogni tic del clock dalla procedura del servizio di interrupt. L'ora sull'orologio (software) sarà sempre corretta grazie a questo metodo.

Se c'è solo un clock e un solo computer, non importa molto se questo clock è sfasato di poco. Poiché ogni processo in esecuzione sulla macchina utilizza lo stesso orologio, la macchina continuerà a mantenere la sua coerenza interna. Ad esempio, se il file input.c ha l'ora 2151 e il file input.o ha l'ora 2150, make ricompilerà il file sorgente anche se l'orologio è sfasato di 2 e i tempi effettivi sono rispettivamente 2153 e 2152. Questo perché *make* utilizza i timestamp per determinare la posizione relativa dei file nella struttura di directory. I tempi relativi sono le uniche cose che contano davvero.

La circostanza subisce un cambiamento drammatico non appena vengono implementate più CPU, ognuna delle quali possiede il proprio orologio. Anche se la frequenza a cui opera un oscillatore a cristallo è tipicamente abbastanza coerente, è impossibile garantire che i cristalli in vari computer funzionino tutti alla stessa identica frequenza. Questo perché diversi computer utilizzano diversi tipi di cristalli. In pratica,

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

quando un sistema ha *n* computer, tutti gli *n* cristalli funzioneranno a velocità leggermente diverse. Ciò farà sì che gli orologi (software) vadano gradualmente fuori sincrono, il che comporterà la restituzione di valori diversi quando vengono letti. Clock skew è il termine usato per descrivere questa differenza nei valori temporali. Come risultato di questa distorsione dell'orologio, i programmi che si aspettano che il tempo associato a un file, oggetto, processo o messaggio sia accurato e indipendente dal computer su cui è stato generato (ad esempio, quale clock è stato utilizzato), possono fallire, come si è visto nell'esempio *make* presentato in precedenza in questa sezione.

L'ora precisa sull'orologio è un'informazione essenziale per alcuni tipi di sistemi, come i sistemi in tempo reale.

In queste particolari circostanze, è richiesto l'uso di clock fisici esterni. I clock fisici multipli sono generalmente considerati desiderabili a causa dei loro benefici percepiti di maggiore efficienza e ridondanza; tuttavia, ciò si traduce in due problemi: (1) Come possiamo sincronizzarli con gli orologi nel mondo reale? (2) Come possiamo sincronizzare gli orologi tra loro?

Lo Universal Coordinated Time, che è più comunemente abbreviato in UTC, è lo standard con cui il tempo viene misurato in tutto il mondo. La sincronizzazione di tutti i moderni orologi civili con UTC è diventata lo standard del settore in tutto il mondo. Una quarantina di stazioni radio a onde corte situate in tutto il mondo inviano ciascuna un breve impulso all'inizio di ogni secondo che è sincronizzato con UTC in modo che le persone che richiedono un tempo preciso possano riceverlo. La precisione di queste stazioni è di circa 1 millisecondo, ma a causa di fluttuazioni atmosferiche casuali che possono influenzare la lunghezza del percorso del segnale, la precisione di queste stazioni non è migliore di ±10 msec nella pratica.

Ci sono diversi satelliti nell'orbita terrestre che forniscono anche il servizio UTC. UTC può essere fornito con una precisione di 0,5 millisecondi dal Geostationary Operational Environment Satellite, e alcuni altri satelliti funzionano in modo ancora più accurato. I server a terra con una precisione di 50 nanosecondi possono essere costruiti combinando le ricezioni da diversi satelliti.

I ricevitori per l'UTC sono prontamente disponibili per l'acquisto e la stragrande maggioranza dei computer moderni ne include uno.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Bibliografia

- M. van Steen and A.S. Tanenbaum, Distributed Systems, 3rd ed., distributed-systems.net, 2017.
- Malone T. and Crowston K. The Interdisciplinary Study of Coordination. ACM Computing Surveys, 1994.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).